

PG220 : Manipulation 1

Hicham LAKHLEF

6 septembre 2026

1 Compilation et exécution d'un fichier Java

Dans cette séance, nous allons découvrir comment compiler et exécuter un fichier de code Java, et effectuer des manipulations basiques sur les objets.

1.1 Étape 1 : Copie du code source

Tout d'abord, veuillez recopier le code source suivant dans un fichier nommé `test.java`. Placez ce fichier dans un répertoire vide créé pour l'occasion (exemple : `pg220/m1`).

```
// Definition de classe, obligatoire dans chaque fichier Java.  
// Le nom de la classe doit tre gal au nom du fichier.  
class test {  
    // Point d'entree des programmes Java. Le tableau  
    // de String args correspond  
    // aux param tres par la ligne de commande.  
    public static void main(String[] args) {  
        // D clARATION d'un objet s de type String.  
        String s;  
        // Cr ation de l'objet s (abord dans la prochaine s ance).  
        s = new String("Hello-World!");  
        // Affichage de l'objet s.  
        System.out.println(s);  
    }  
}
```

1.2 Étape 2 : Compilation et exécution

Pour compiler ce fichier, il faut faire appel au compilateur Java : `javac`. Vous pouvez compiler le fichier avec la commande suivante : `javac test.java`.

Si tout se passe bien, vous devriez obtenir un fichier nommé `test.class`. Il contient le code source précédent sous forme de bytecode. Pour lancer le fichier, il faut cette fois passer par l'intermédiaire de la machine virtuelle Java, avec la commande suivante `java test`.

Normalement, l'exécution du programme devrait afficher la chaîne **Hello World!** sur votre console.

2 Modification du programme

Consultez la javadoc de la classe **String**
(<https://docs.oracle.com/javase/7/docs/api/java/lang/String.html>).

1. Cherchez la méthode qui permet de mettre une chaîne en **majuscules**.
 - Comment fonctionne cette méthode ?
 - Modifiez le programme précédent pour construire une chaîne en majuscules à partir de **s**.
2. De la même manière, trouvez les méthodes qui permettent de :
 - (a) Convertir une chaîne en **minuscules**.
 - (b) Extraire une **sous-chaîne** à partir d'une chaîne.
 - (c) Vérifier si une chaîne **commence par** une autre chaîne donnée.
 - (d) Calculer la **longueur** d'une chaîne.
 - (e) **Concaténer** deux chaînes.
3. Testez toutes ces méthodes dans un programme Java.
4. Question de réflexion : Les méthodes qui renvoient des **String** modifient-elles l'objet sur lequel on les appelle, ou bien travaillent-elles sur des **copies** ?

3 Comparaison des valeurs primitives et des objets en Java

3.1 Objectif

Comprendre les différences de comportement de l'opérateur **==** lors de la comparaison de types primitifs et d'objets en Java.

3.2 Partie 1 : Comparaison de types primitifs

Soit le programme suivant :

Code :

```
int i1 = 12;
int i2 = 12;
System.out.println(i1 == i2);
```

Quelle sera la sortie de ce programme ? Expliquez pourquoi.

Soit le programme suivant :

```
double d1 = 12.0;
double d2 = 12.0;
System.out.println(d1 == d2);
```

Quelle sera la sortie de ce programme ? Justifiez votre réponse.

Étudiez le code suivant impliquant des objets Integer et prédisez les sorties.

```
Integer obj1 = new Integer(12);
Integer obj2 = new Integer(12);
System.out.println(obj1 == obj2);
System.out.println(obj1.equals(obj2));
```

Quelle sera la sortie du programme suivant ? Justifiez votre réponse

```
String str1 = new String("hello");
String str2 = new String("hello");
System.out.println(str1 == str2);
System.out.println(str1.equals(str2));
```

3.3 Partie 3 : Analyse comparative

3.4 Différences entre == et equals

- En quoi la comparaison de types primitifs diffère-t-elle de celle des objets lorsque l'on utilise l'opérateur == ?
- Proposez un scénario dans lequel une mauvaise compréhension de ces concepts pourrait entraîner un bogue dans un programme Java. Décrivez comment ce bogue pourrait être évité.

4 Création d'une classe

- Essayez maintenant de faire une classe class Point qui comporte deux attributs int x et int y, ainsi qu'un constructeur et des accesseurs/mutateurs pour tous les attributs. Ajouter dans cette classe un point d'entrée qui crée un point et affiche ses coordonnées.
- Modifiez la classe Point pour qu'elle implémente une méthode distance qui prend un autre point en entrée et retourne la distance euclidienne entre les deux points.